Programmeerproject 2 2026-2027

Titularis: prof. dr. Antonio Paolilo
Begeleidende assistenten: ir. Youri Coppens & ir. Louise Van Verre
Contact: project2ba@dinf.vub.ac.be

Inhoudsopgave

1. Inleiding . . . 1
2. Projectbeschrijving . . . 2
3. Functionele vereisten . . . 2
   3.1. Fase 1 . . . 3
   3.2. Fase 2 . . . 4
   3.3. Fase 3 . . . 5
4. Niet-functionele vereisten . . . 7
   4.1. Software-architectuur en codekwaliteit . . . 7
   4.2. Tests . . . 8
5. Deliverables . . . 9
   5.1. Code . . . 9
   5.2. Documenten . . . 9
      5.2.1. Handleiding . . . 9
      5.2.2. Design document . . . 10
   5.3. Hoe inleveren? . . . 10
6. Demonstratie en verdediging . . . 11
7. Beoordeling . . . 11
   7.1. Puntenverdeling . . . 11
   7.2. Regeling voor laattijdig indienen . . . 11
   7.3. Deelnemingsvereisten . . . 11
   7.4. Verdedigings-ondergrens (per fase) . . . 12
   7.5. Generatieve-AI-beleid . . . 12
   7.6. Tweede zittijd . . . 12
8. Hints en tips . . . 13
A. Jaaroverzicht (deadlines) . . . 14

1. Inleiding

In Programmeerproject 1 werd er voor het eerst in jullie opleiding Computerwetenschappen een groot programma in Scheme gebouwd. Voor Programmeerproject 2 maken jullie individueel een complete softwaretoepassing. Deze bevat een softwarearchitectuur met meerdere componenten, inclusief een databank. Daarenboven zal je uiteindelijke “product” niet enkel bestaan uit een werkend programma, maar ook de nodige documentatie moeten bevatten.

Dit project zal gequoteerd worden op drie vlakken:

- Het ontwikkelen van een programma van grote omvang dat moet voldoen aan een reeks gegeven functionele en kwaliteitseisen.
- Het ontwerpen, testen en documenteren van een softwareproject, dat blijk geeft van een grondige analyse van de gegeven opdracht.
- De schriftelijke en mondelinge rapportering van het ontwikkelingstraject. Dit bestaat uit een handleiding gericht op de eindgebruiker, een design document gericht op programmeurs, een demonstratie van het programma en een mondelinge verdediging.

Dit document bevat een algemene omschrijving van het project (Sectie 2), de minimale functionele en niet-functionele vereisten (Sectie 3 en Sectie 4), en details over de deliverables (Sectie 5). Verder wordt in dit document het verloop van de mondelinge verdedigingen toegelicht (Sectie 6), de evaluatie van het project beschreven (Sectie 7) en kan je nog wat praktische tips terugvinden (Sectie 8). Ter afsluiting vind je een jaaroverzicht in Appendix A. In dit project is er veel ruimte gelaten om je creativiteit op bot te vieren. Je wordt aangemoedigd om de originele opdracht te verrijken met eigen inbreng die in het thema van de opdracht past.

2. Projectbeschrijving

Momenteel bestaat er een uitgebreid gamma aan applicaties om nieuwe sociale contacten te leggen. Voorbeelden hiervan zijn Tinder, Bumble, Hinge, Grindr enz. In deze applicaties maken gebruikers een profiel aan met informatie over zichzelf (bv.: leeftijd, hobby's, type relatie die ze willen). Dit profiel wordt getoond aan andere gebruikers van de app die vervolgens hun interesse kunnen aangeven door bijvoorbeeld links (*dislike*) of rechts (*like*) te swipen. Als beide partijen interesse tonen in mekaar, komt er een match tot stand. Gematchte gebruikers kunnen vervolgens met elkaar communiceren en afspreken.

Het doel van het project is om een vereenvoudigde dating-applicatie te implementeren en zal uiteindelijk bestaan uit een aantal componenten:

1. Een **client-applicatie** waarmee een gebruiker een profiel met hun gegevens en voorkeuren kan invoeren en van het systeem aanbevelingen kan ontvangen. Iedere gebruiker dient zijn eigen instantie van deze applicatie te hebben;
2. Een **server-applicatie** die de gegevens van alle gebruikers verwerkt en toelaat profielen te matchen met mekaar. De communicatie tussen de server met de clients zal uiteindelijk via een netwerkverbinding (TCP) dienen te verlopen;
3. Een onderliggende **relationele databank** die de gebruikersgegevens bewaren zal.

3. Functionele vereisten

Dit project bestaat uit een aantal **basisvereisten** die je verplicht moet implementeren. Deze vereisten worden verdeeld over drie fases waarin je het project zal implementeren en opleveren. We beschrijven hieronder per fase de verwachte vereisten in detail. Er dient incrementeel gewerkt te worden: je dient dus je project zodanig te ontwikkelen dat het makkelijk uit te breiden of aan te passen blijft naarmate de fases vorderen. Daarom begint het project met een korte voorstudie in het begin van het academiejaar tijdens

een WPO onder begeleiding van de assistenten. Deze voorstudie dient voor jezelf om een initieel ontwerp van je project vast te leggen.

3.1. Fase 1

In de eerste fase implementeer je de fundamentele structuur van het project: de basiscomponenten om verscheidene clients met een minimale grafische gebruikersinterface en een server op te stellen. De server- en de client-component moeten duidelijk gescheiden onderdelen in het project zijn.

1. **Grafische Gebruikersinterface: Profiel Aanmaken**

   Het project moet een grafische gebruikersinterface (GUI) aanbieden. Hiervoor moet je de "Racket Graphical Interface Toolkit" gebruiken. Zorg er voor dat je GUI modulair is: de implementatie van de functionaliteit van je programma moet gescheiden zijn van de grafische interface. Een goede vuistregel: stel dat je een andere GUI moet toevoegen (bv. voor een tablet of smartphone), dan zou je de code die de functionaliteit van je programma implementeert niet hoeven aan te passen, maar enkel de code die de GUI tekent.

   In fase 1, zal de GUI de eindgebruiker moeten toelaten om een profiel aan te maken. Dit betekent dat de gebruiker persoonlijk informatie kan invoeren met behulp van intuïtieve grafische elementen. De minimale verwachte data is een naam, leeftijd, hobby's en het type relatie waarnaar gezocht wordt. Je kan hier jouw creativiteit de vrije loop geven en dit uitbreiden met extra gegevens zoals een kleine beschrijving of foto's. De ingevoerde profieldata dient verstuurd te worden naar het server-component en hier bewaard te worden zolang de server actief is.

2. **Meerdere Gebruikers**

   Voor een dating-app is het belangrijk dat er meer dan 1 gebruiker is. Daarom is het noodzakelijk om vanaf fase 1 hier rekening mee te houden. Het project zal dus met meerdere users moeten kunnen omgaan. Er moet een **minimum van 4 users tegelijkertijd** kunnen runnen, met elk een eigen instantie van de GUI. Belangrijk is natuurlijk dat iedere gebruiker zelf een profiel kan aanmaken en die gegevens correct verstuurd en bewaard worden op de server.

3. **Client-Server Architectuur: Basis**

   Zoals vermeld in Sectie 2, zal het project bestaan uit enkele componenten, waaronder een client en een server. Vanaf fase 2 zullen deze twee componenten met elkaar communiceren via TCP. Om grote aanpassingen aan jouw ontwerp in die fase te vermijden is het belangrijk om vanaf het begin de code te organiseren in die twee componenten. De server zal dus verantwoordelijk zijn voor het bewaren van profielgegevens en de client voor het aanbieden van een GUI voor eindgebruikers om (initieel) een profiel aan te maken. Tijdens de eerste fase volstaat het dat profielgegevens in het centraal werkgeheugen bewaard blijven zolang de server actief is.

3.2. Fase 2

In de tweede fase vervolledig je de splitsing van je server en clients door deze componenten in aparte processen te draaien en met elkaar te laten communiceren via een TCP-verbinding. Verder dien je in fase 2 de gebruikersinterface van je client-applicatie uit te breiden met login-functionaliteit, alsook een databank toe te voegen aan je server-component.

4. **Grafische Gebruikersinterface: Login**

   In fase 1, kan de eindgebruiker in de GUI reeds een profiel aanmaken. De eindgebruiker moet echter elke keer bij het opstarten van het systeem opnieuw een profiel aanmaken. Daarom zal in fase 2 de GUI uitgebreid moeten worden met een login-functionaliteit. Hierdoor zal de gebruiker kunnen kiezen tussen een nieuw profiel aanmaken of een bestaand profiel gebruiken. We verwachten de volgende minimale vereisten:
   - De eindgebruiker moet de mogelijkheid hebben om bij opstart van de client-applicatie te kiezen om in te loggen met een bestaand profiel of om een nieuw profiel te registreren.
   - Inloggen vereist een gebruikersnaam en wachtwoord.
   - Na te hebben ingelogd, kan je gegevens van het profiel aanpassen, alsook terug uitloggen.
   - Na te hebben uitgelogd, zou het niet meer mogelijk mogen zijn om profielgegevens te zien noch aan te passen in het systeem.

5. **Persistente data**

   In een dating-app is het profiel van een gebruiker gelinkt aan een account die op de server bewaard wordt. Hierdoor moeten de gebruikers niet altijd opnieuw een profiel aanmaken om gebruik te maken van de applicatie. Bepaalde gegevens van een account blijven constant, terwijl andere gegevens aanpasbaar zijn. Om dit te verwezenlijken wordt deze data normaal bijgehouden in een databank. In fase 2 is het dus de bedoeling dat je server-component wordt uitgebreid met een relationele databank om de profielgegevens van gebruikers te bewaren. Hiervoor moet er gebruik gemaakt worden van SQLite en de Racket DB-plugin. Tijdens één van de contactmomenten voorzien we een tutorial die het gebruik van SQLite en de Racket DB-plugin zal toelichten. Je dient in het server-component een abstractielaag te voorzien die minstens de volgende CRUD-operaties voorziet op je databank:
   - Het aanmaken van een SQLite database-file indien deze nog niet bestaat;
   - Het definiëren van de nodige relaties (tabellen) en hun constraints (primary keys, foreign keys etc.) in de voorziene database-file;
   - Query's om gebruikers en hun gegevens op te slaan;
   - Query's om gebruikersgegevens op te vragen die weergegeven dienen te worden in de client-applicatie;
   - Query's om gebruikersgegevens aan te passen;
   - Query's om gebruikers te verwijderen;

In je database voorzie je **minstens** 5 profielen met voldoende variatie in profielgegevens en steunt hierop om je code te demonstreren.

6. **Client-Server Architectuur: Aparte programma's / TCP**

   In fase 2, worden zowel clients als de server elk een apart proces/programma met elk hun eigen geheugen. Dit betekent dat alle gegevens die ze willen delen, expliciet moeten uitgewisseld worden via het netwerk, wat zal verlopen met een TCP-verbinding. Dit laat eveneens toe om beide componenten op aparte machines te draaien (bv. iedere client op een andere laptop en de server in een datacenter). Ook het design van de API tussen beide componenten is van cruciaal belang: welke berichten moeten uitgewisseld worden en welke informatie moeten deze bevatten? Na het toevoegen van een TCP-verbinding in je project, moet het dus minimaal mogelijk zijn om je clients en server apart op te starten op je eigen computer en ze te laten communiceren met behulp van het lokale IP-adres (localhost). Idealiter, zou je design het moeten toelaten om makkelijk de server op een andere machine op hetzelfde lokale netwerk (LAN) te draaien, en je clients daarmee te kunnen laten verbinden.

3.3. Fase 3

In de derde en laatste fase implementeer je het algoritme dat de server toelaat om profielen te koppelen aan mekaar, alsook de nodige grafische features in de client-applicatie om voorgestelde profielen te (dis)liken. Als kers op de taart voeg je nog een gekozen uitbreiding toe.

7. **Grafische Gebruikersinterface: Matching**

   In fase 3 zal het project moeten toelaten dat eindgebruikers andere gebruikers hun profiel zien, liken of disliken. De GUI zal hiervoor dus moeten worden uitgebreid. Volgende minimale vereisten worden verwacht:
   - Een eindgebruiker kan voorgestelde profielen van andere gebruikers bekijken.
   - De GUI bevat een mechanisme om een voorgesteld profiel te liken of disliken.
   - Eindgebruikers krijgen een notificatie als ze elkaars profiel hebben geliked. De notificatie verschijnt bij beide partijen, ofwel meteen als de gebruiker ingelogd is, ofwel nadat de gebruikt ingelogd is.

8. **Recommendation**

   In een dating app worden er mogelijke matches aanbevolen aan een eindgebruiker. In fase 3, moet het project dus worden uitgebreid met een algoritme voor aanbevelingen. Deze aanbevelingen houden rekening met de eisen van de eindgebruiker. De eindgebruiker wil bijvoorbeeld enkel profiel zien tussen een bepaalde minimum- en maximumleeftijd. De minimale vereisten voor dit algoritme zijn:
   - Aanbevolen matches moeten allemaal de juiste leeftijdscategorie hebben.
   - De aanbevolen matches mogen geen eerder gedislikete users bevatten.
   - Het algoritme moet er rekening mee houden dat het systeem dynamisch is: er kunnen nieuwe users bijkomen en oude users weggaan.

Je algoritme steunt op de accountgegevens in de databank om matches tussen 2 gebruikers voor te stellen. Deze voorstellen dienen rekening te houden met eerder gemaakte voorstellen, zodat een gebruiker telkens nieuwe voorstellen/matches verkrijgt. Het systeem dient dus te onthouden welke profielen reeds voorgesteld waren in het verleden, en wat de gebruiker toen koos (like of dislike).

Je mag dit algoritme ook verder uitbreiden met andere vereisten. Zo zouden matches ook aanbevolen kunnen worden gebaseerd op gedeelde voorkeuren zoals hobby's en zo kunnen aanbevelingen ook gerangschikt worden van meest gedeelde voorkeuren naar minst.

9. **Extra uitbreiding**

   Kies uit de lijst hieronder één uitgebreide vereiste die je implementeert in fase 3. Het is toegestaan om meerdere uitbreidingen te maken, maar doe dit slechts zodra je minstens één uitbreiding volledig **werkende** hebt gekregen! Geef dus duidelijk aan in je documentatie (zie Sectie 5) welke uitbreiding primeert in je project: geef aan hoe de functionaliteit van je uitbreiding gebruikt kan worden door eindgebruikers in je handleiding, alsook hoe de uitbreiding geïntegreerd werd in het design document.

   Mogelijke features die toegevoegd kunnen worden zijn:

   - **Maybe like**: In plaats van een profiel te liken of disliken, kan een eindgebruiker nu ook *maybe-liken*. Dit is een extra optie dat aangeeft dat je het aanbevolen profiel nog niet meteen wenst goed of af te keuren, en bijgevolg dus niet wegfiltert uit de toekomstige lijsten van aanbevolen matches. Hierdoor kan het maybe-gelikete profiel later nog een keer terugkeren als aanbevolen match.
   - **Activiteit Matching**: Niet alle activiteiten kan je alleen doen, bijvoorbeeld tennis of basketbal. Om mensen te helpen, wordt er verwacht dat je jouw applicatie uitbreidt met activiteiten. Deze activiteiten worden aangemaakt door eindgebruikers die op zoek zijn naar anderen om hen te vergezellen in de activiteit. Merk op dat er gelet moet worden dat reeds bestaande activiteiten niet gedupliceerd worden. Gelijkaardig aan profielen zullen andere eindgebruikers deze activiteiten aanbevolen krijgen. Dan swipen ze om niet/wel deel te nemen aan de activiteit.
   - **P2P Chat**: Wanneer twee eindgebruikers elkaar liken, krijgen beide gebruikers een notificatie. In bestaande dating-apps zoals Tinder en Bumble kunnen deze gebruikers vervolgens ook communiceren via chat. In deze vereiste wordt er dus verwacht dat je jouw applicatie uitbreidt met een peer-to-peer chat feature. De betrokken clients openen een rechtstreekse communicatielijn met mekaar waarover berichten verstuurd en gelezen kunnen worden, zonder dat deze via de server passeren. De chatfunctie dient geïntegreerd te zijn in de GUI.
   - **Security**: Gelekte gebruikersdata kunnen nefaste gevolgen hebben voor bedrijven (bv. boetes en reputatieschade). Het is dus belangrijk dat er veilig om wordt gegaan met gebruikersdata. In deze vereiste wordt er dus verwacht dat je jouw applicatie uitbreidt om de gebruikersdata in de database te beveiligen.

Wachtwoorden van profielen dienen geëncrypteerd opgeslagen te worden in de databank, en het inlogproces zou beveiligd moeten zijn: op z'n minst zouden wachtwoorden niet als leesbare tekst verstuurd mogen worden van de client naar de server. Je dient dus opzoekwerk te verrichten in het voorzien en toepassen van veilige encryptie- en uitwisseling-technieken in je project. Merk op dan men heel ver kan gaan in het beveiligen van softwaresystemen, maar er wordt zeker niet verwacht dat je dit tot het uiterste toepast in je project. Je kan bijvoorbeeld steunen op een hardgecodeerde sleutel voor bepaalde versleutelingstechnieken in plaats van het gekozen protocol tot op de letter exact te implementeren. Het doel van deze uitbreiding is je te verdiepen in de mogelijkheden en een basisprincipe uit te werken.

Zorg er voor dat je je security-systeem duidelijk documenteert in het design document: beschrijf hoe en wat er werd toegepast qua beveiligingsmaatregelen.

- **Raspberry Pi**: Volgens de minimale functionele vereisten draai je zowel clients als server op je laptop. Een mogelijke uitbreiding is om client(s) op één of meerdere laptops te draaien en de server op een Raspberry Pi. Een Raspberry Pi is een mini-computertje, waarop je Linux kan installeren en dat je met het netwerk kan verbinden. Je laptop communiceert met de Raspberry Pi via een TCP-verbinding op het lokale netwerk. Je dient voor deze uitbreiding **verslag te geven** in je documentatie over de zaken die ondernomen diende te worden om de server werkende te krijgen op de Raspberry Pi, alsook een **demonstratie te filmen** van je project gebruikmakende van de Raspberry Pi. Deze opname dient mee opgeleverd te worden met je inzending van fase 3. Indien je een Raspberry Pi wilt gebruiken, kan je er één lenen door ons te mailen. Merk op dat er slechts een beperkte voorraad aan Raspberry Pi's beschikbaar is dus vraag dit tijdig aan!
- **Andere voorstellen:** Moest je zelf een uitdagende uitbreiding willen toevoegen aan je project, bestaat hier ook de mogelijkheid voor. Stuur **ten laatste vóór het einde van de lentevakantie (11/04/2027 23u59)** een mailtje naar de assistenten (project2ba@dinf.vub.ac.be) om je voorstel te bespreken, en **na goedkeuring** kan je je eigen idee implementeren.

4. Niet-functionele vereisten

Bij het ontwikkelen van een groot softwareproject is het belangrijk dat je de kwaliteit van je code niet uit het oog verliest. Bijgevolg zullen we rekening houden met de codekwaliteit bij het evalueren van je project. Een groot project kan snel leiden tot ingewikkelde en moeilijk begrijpbare code. Je project opsplitsen in logische modules, het gebruik van goede ADT's, en het ontwerp van goede API's kunnen dit voorkomen. Daarnaast wordt er verwacht dat je je programma test door middel van "unit tests".

4.1. Software-architectuur en codekwaliteit

Algemene informatie omtrent de kwaliteitseisen vind je in het document **"Kwaliteitseisen Programmeerproject"** op Canvas. **Lees dat document grondig door!** Hou

rekening met elke eis die in dat document staat. Bovendien zie je in de cursussen “Structuur van Computerprogramma’s 1” alsook “Algoritmen en Datastructuren 1, 2 en 3” hoe goede Scheme-code, met gebruik van modules, er uitziet.

Enkele tips specifiek voor dit project:

- Kies bij elk ADT de gepaste abstractietechniek en implementatie.
- Splits je programma op in modules: logische onderdelen met elk een eigen specifieke functionaliteit. Een software-ontwerp met goed gekozen modules zal je veel tijd besparen bij het ontwikkelen van je project, en je project robuuster en begrijpelijker maken.
- Denk bij grote ADT’s na of ze niet kunnen gesplitst worden in kleinere, lossere onderdelen. Kleine ADT’s die bijna altijd samenwerken kan je dan weer trachten samen te voegen. Dit helpt bijvoorbeeld om de grafische interface gescheiden te houden van de rest van je programma. Kleine veranderingen aan je GUI dienen weinig tot geen impact te hebben op de rest van je programma.
- Denk goed na over de interfaces (API’s) tussen de verschillende componenten. Hou rekening met de mogelijkheid dat deze op verschillende machines zullen draaien.
- Een goede voorstudie zal je al een heel eind op weg brengen.

4.2. Tests

Unit testing is een techniek waarbij softwaremodules of stukjes broncode (“units”, bv. een module of een ADT) afzonderlijk getest worden. Bij unit testing worden voor iedere unit verschillende tests ontwikkeld, die alle functionaliteit van de unit verifiëren. Deze doorlopen een aantal test cases, waarbij telkens de echte output wordt afgetoetst tegen een verwachte waarde. Het doel van unit testing is om functionele units onafhankelijk van elkaar te testen op correcte werking. Zo weet je in welk deel van je programma een fout is geslopen. Het is niet nodig elke kleine functie te testen: focus in de plaats op het testen van blokken functionaliteit.

We verwachten in elke fase dat je unit tests ontwikkelt voor de cruciale functionaliteit van je project. Telkens je iets verandert aan je code, kan je de overeenkomstige unit tests doorlopen om te controleren of alles nog werkt. Een batterij aan tests zal van onschatbare waarde zijn wanneer je je code aanpast naar het einde van het project toe. Je moet deze tests ook toevoegen aan de code die je inlevert. Je gebruikt hiervoor best de RackUnit bibliotheek. Tijdens één van de contactmomenten voorzien we een tutorial om het gebruik van deze bibliotheek te illustreren.

In het ideale geval is elke test case onafhankelijk van de rest van het systeem. Eventueel worden hiertoe stukken code geschreven die andere stukken van je programma “simuleren”, om zo één stuk te kunnen testen zonder afhankelijkheden op een ander stuk. Dit noemt *mocking*, en kan nuttig zijn om bijvoorbeeld tests te schrijven die onafhankelijk zijn van netwerkcommunicatie.

In het document **“Kwaliteitseisen Programmeerproject”** op Canvas vind je nog wat extra duiding over hoe je je project best test en debugt tijdens het ontwikkelen.

5. Deliverables

Voor elke fase verwachten we een aantal deliverables. Deze sectie beschrijft wat je exact moet indienen. De deadlines voor je opleveringen vind je terug in Appendix A.

5.1. Code

Alle broncode van je toepassing, inclusief tests, dien je op te sturen. Zorg ervoor dat deze uitvoerbaar is vanuit DrRacket. Over het algemeen dient je project dus in de programmeertaal Racket[1] of een geziene Scheme-standaard (bv. R^{7}RS) geschreven te worden.

Code bestaat uit Racket-files (`.rkt`) en — vanaf fase 2 — de database file. **In je database voorzie je vanaf fase 2 minstens 5 profielen met voldoende variatie in profielgegevens en steunt hierop om je code te demonstreren.**

Documenteer je code zodat deze gemakkelijk te begrijpen is. Laat geen debugging-code (bv. `print`-statements) of ongebruikte code (zowel in commentaar als losstaande procedures) staan. Dit laatste maakt je broncode namelijk slordig en schaadt de leesbaarheid. Verwijder ook (automatisch gegenereerde) backupbestanden en gecompileerde code, o.a. bestanden die eindigen met `~`, `.bak` of `.zo`, de mappen `compiled`...

5.2. Documenten

Naast de code worden er 2 bijhorende documenten verwacht in je oplevering. In wat volgt beschrijven we meer in detail wat er per document qua inhoud verwacht wordt. Voorbladen voor deze documenten zijn niet nodig: een hoofding met een titel, je **naam**, je **studentennummer** en **email-adres** volstaan. Gebruik voor alle documenten de automatische spellingscontrole van je tekstverwerker en lees ze grondig na om schrijffouten te voorkomen.

5.2.1. Handleiding

Een handleiding van je toepassing dient gericht te zijn op **eindgebruikers**, d.w.z. voor mensen die geen technische achtergrondkennis van zaken hebben. De handleiding dient minstens volgende aspecten te bevatten:

- Een algemene omschrijving van je toepassing.
- Hoe deze te installeren en te starten: welke libraries moeten we installeren, welk Racket-bestand moeten we starten, wordt er bepaalde input verwacht etc.
- Een beschrijving van alle functionaliteit en hoe deze gebruikt kan worden in de GUI, bv. het aanmaken van profielen of het matchen met andere profielen. Vergeet je uitbreiding ook niet!

Voorzie screenshots die de functionaliteit illustreren. Beperk dit document tot **max. 10 pagina's** in zijn geheel (alles inbegrepen), minimum 5.

[1] Gebruik een versie van Racket hoger of gelijk aan 8.0.

5.2.2. Design document

Het design document dient de architectuur van het project te omschrijven: uit welke componenten bestaat het project, welke abstracties (ADT's) werden hiervoor opgesteld, en een beschrijving van de verantwoordelijkheden en functionaliteit van iedere abstractie. Dit document dient gericht te zijn op **ontwikkelaars**, d.w.z. mensen mét technische kennis. Het is bijgevolg dan ook een technische specificatie die het ontwerp en implementatiekeuzes van het project beschrijft en motiveert.

Het design document moet volgende zaken bevatten:

- Een algemene omschrijving van je toepassing. Zeg hierbij ook welke van de uitgebreide vereisten je uitgekozen hebt.
- Een beschrijving van de gehele software-architectuur. Maak gebruik van o.a. een afhankelijkheidsdiagram om de verschillende componenten, ADT's en hun relaties te tonen. Leg in het algemeen uit hoe de verschillende componenten zich tot mekaar verhouden. (Je begint best hiermee voor je de componenten apart bespreekt.)
- Een beschrijving van elke component apart. Leg uit wat de taken of functionaliteiten van elk component zijn. Je kan hierbij de ADT's van de component oplijsten, maar je moet niet per se elke functie van elk ADT oplijsten: focus eerder op een algemene beschrijving van het ADT. Vermeld zeker ook de algoritmes die je gebruikt (bv. voor het matchen van profielen). Als dit bestaande algoritmes zijn, geef je hun naam en eventueel een bronvermelding; als je een bestaand algoritme hebt aangepast, leg dan je aanpassingen uit; als je volledig zelf een algoritme hebt bedacht, leg dan uit hoe het werkt. Relevante implementatiedetails die noodzakelijk zijn om de implementatie te snappen van een onderdeel dien je eveneens te vermelden.
- Een beschrijving van de interfaces of communicatieprotocollen tussen de client en server.
- **Vanaf fase 2** moet er ook een (E)ER model en bijhorend database-schema aanwezig zijn die de structuur van je databank formeel beschrijft.

Gebruik waar nodig een diagram en licht deze toe. Voorzie bijvoorbeeld legendes die bepaalde symbolen beschrijven. Je design document zou allesomvattend moeten zijn: het lezen van dit document zou moeten volstaan om je code te kunnen begrijpen. Stel je bij het schrijven van je document voor dat iemand die nu in derde bachelor zit in de zomervakantie aan de slag moet kunnen met jouw project.

Beperk dit document tot **max. 10 pagina's** in zijn geheel (alles inbegrepen).

5.3. Hoe inleveren?

Converteer alle documenten naar **PDF-formaat**. We aanvaarden geen Word, LibreOffice, LaTeX, Typst of andere bestanden. Steek al je documenten en code (ook die van derden) in een mapje met je voor- en achternaam (bv. `jan-peeters`), en maak hiervan een zip-archief met dezelfde naam. Zorg voor een overzichtelijk project door je documenten en bestanden in mappen te verdelen. Upload dit via Canvas vóór de deadline.

6. Demonstratie en verdediging

Voor fases 2 en 3 organiseren we een mondelinge verdediging. De verdediging duurt voor elke student 15 minuten en bestaat uit de volgende onderdelen:

1. **7 minuten presentatie.** Hiervoor raden we aan een presentatie te maken van zo'n vijf tot tien slides. Zorg er voor dat je zeker diagrammen (bv. afhankelijkheidsdiagram, (E)ER model en DB-schema) uit je design documenten toont. Leg aan de hand hiervan uit welke componenten je programma bevat, wat hun functionaliteit is, en hoe ze met elkaar communiceren.
2. **3 minuten demonstratie.** Let op de tijd: je hoeft niet alle functionaliteit apart te demonstreren, toon liever één ingewikkeld scenario dat alle vereisten demonstreert. Test dit zeker op voorhand!
3. **5 minuten vragen.** Hierbij zullen we je enkele vragen stellen op basis van je handleiding, design document en presentatie. Ook bestaat de kans dat er vragen over je implementatie (code) gesteld worden.

De slides van je presentatie dien je vóór aanvang van de verdediging in PDF-formaat door te mailen naar project2ba@dinf.vub.ac.be.

Tijdens de verdediging voorzie je **zelf** een laptop om te presenteren en demonstreren.

7. Beoordeling

In deze sectie beschrijven we hoe de evaluatie voor dit vak verloopt. Voor alle details verwijzen we door naar de studiedeelfiche.

7.1. Puntenverdeling

Het project is opgebouwd in drie fases. Elke fase wordt gequoteerd. Het eindcijfer bestaat uit:

- **60%** voor de functionaliteit en de kwaliteit van de code;
- **40%** voor de kwaliteit van de rapportering (handleiding, design document) en (vanaf fase 2) de mondelinge verdedigingen.

7.2. Regeling voor laattijdig indienen

Inzendingen na de deadline van een fase **verliezen twee punten per kalenderdag** op het cijfer van die fase. Een inzending die **meer dan vier kalenderdagen te laat is, wordt geregistreerd als afwezig**.

7.3. Deelnemingsvereisten

De student dient alle drie de fases in te leveren en beide mondelinge verdedigingen bij te wonen. Het ontbreken van een fase-inzending of een mondelinge verdediging wordt geregistreerd als AFW (afwezig) voor de hele cursus. Aanwezigheid op de hoorcolleges (HOC) en werkcolleges (WPO) is sterk aanbevolen maar niet individueel verplicht; het project veronderstelt wel dat de student de bijhorende leerstof heeft doorgewerkt.

7.4. Verdedigings-ondergrens (per fase)

Indien de student strikt minder dan 8/20 behaalt op de mondelinge verdediging van een bepaalde fase (2 of 3) terwijl het gewogen fase-cijfer (code, documentatie en verdediging gecombineerd) 8/20 of hoger is, wordt het cijfer van de verdediging het eindcijfer voor die fase (de cijfers voor de code en de documentatie van die fase worden in dat geval niet meer meegerekend). Indien het gewogen fase-cijfer zelf reeds onder de 8/20 ligt, blijft dat het cijfer voor die fase.

7.5. Generatieve-AI-beleid

Het gebruik van generatieve-AI-hulpmiddelen, inclusief maar niet beperkt tot ChatGPT, GitHub Copilot, Claude, Gemini, of enige andere large-language-model-assistent, is **NIET toegestaan** voor de projectcode, voor de inhoud van de handleiding, het design document en het voorbereidingsmateriaal voor de mondelinge verdedigingen.

Uitzondering. Generatieve-AI-gebaseerde **grammatica- en spellingscheckers** op de geschreven documentatie (handleiding en design document) zijn toegestaan, op voorwaarde dat de student in het document vermeldt welke tool(s) gebruikt werden.

Inzendingen die dit beleid schenden, of die de vereiste vermelding weglaten wanneer grammatica-hulpmiddelen gebruikt werden, worden behandeld als academische fraude overeenkomstig het examenreglement van de VUB.

7.6. Tweede zittijd

- Er is geen herhaling van de drie fases in de tweede zittijd. Studenten leveren **één geconsolideerde inzending** in (code en documentatie) die voldoet aan de vereisten van alle drie de fases van de eerste zittijd, en verdedigen het project in **één mondelinge verdediging**.
- **Deelcijfers worden NIET overgedragen van de eerste naar de tweede zittijd.** Het volledige project wordt opnieuw beoordeeld tegen de vereisten van het huidige academiejaar. De student mag zijn code uit de eerste zittijd hergebruiken als uitgangspunt.
- De puntenverdeling (60% code / 40% rapportering + verdediging) blijft ongewijzigd. De verdedigings-ondergrens is van toepassing op de enige verdediging van de tweede zittijd: indien de student strikt minder dan 8/20 behaalt op die verdediging terwijl het gewogen cursuscijfer 8/20 of hoger is, wordt het cijfer van de verdediging het eindcijfer voor de cursus (de cijfers voor code en documentatie worden in dat geval niet meer meegerekend); indien het gewogen cursuscijfer zelf reeds onder de 8/20 ligt, blijft dat het eindcijfer.

De regeling voor laattijdig indienen van de eerste zittijd is niet van toepassing; één enkele deadline voor de tweede zittijd wordt op Canvas gecommuniceerd.

8. Hints en tips

- Leg een eigen deadline vast die bijvoorbeeld één of twee weken voor de echte deadline ligt. Zo zorg je er voor dat je code zeker op tijd af is en dat je nog tijd genoeg hebt om een goede documentatie document te schrijven: deze documenten zijn het startpunt om je project te evalueren. Onderschat de tijd voor het schrijven van je handleiding en design document niet.
- Er wordt veel aandacht besteed aan het ontwerp en het gebruik van ADT's. Een project zonder ADT's of met slechte ADT's is indicatief voor een slecht project. Probeer dit al goed te krijgen tijdens je voorstudie, maar vergeet ook niet dat je later nog mag bijsturen indien nodig.
- Je mag je documentatie in het Nederlands of het Engels schrijven, maar let op taalfouten. Lees grondig na en gebruik de automatische spellingscontrole van je tekstverwerker. Verslagen met veel schrijffouten worden niet geduld.
- Bij vragen of problemen kan je altijd terecht bij de projectbegeleiders via het e-mailadres **project2ba@dinf.vub.ac.be**. Via dit e-mailadres kan je ook een afspraak maken.

A. Jaaroverzicht (deadlines)

Onderstaande tabel geeft aan in welke weken van de academische kalender er bepaalde contactmomenten zullen zijn alsook de deadlines voor opleveringen. Bij wijzigingen of veranderingen zal hierover gecommuniceerd worden op Canvas: het is dus ook van belang de aankondigingen in de gaten te houden!

Week 2 **21/09/2026 09:00**	HOC: Voorstelling project
Week 4	WPO: Voorstudie
Week 6	HOC: GUI Tutorial
Week 7	WPO: Unit Testing Tutorial
Week 11 **29/11/2026 23:59**	**Indienen fase 1 (code en documentatie)**
Week 14	HOC: Feedback fase 1
Week 22	WPO: Database + TCP tutorial
Week 26 **14/03/2027 23:59**	**Indienen fase 2 (code en documentatie)**
Week 28	Mondelinge verdediging fase 2
Week 32	HOC: Feedback fase 2
Week 36 **23/05/2027 23:59**	**Indienen fase 3 (code en documentatie)**
Examenperiode juni	Mondelinge verdediging fase 3

De data voor de tweede zittijd worden gecommuniceerd na het beëindigen van de eerste zittijd.